src\screens\FeedScreen.js

```
import { memo, useCallback, useMemo, useState } from 'react';
import {
    Dimensions,
    FlatList,
    RefreshControl,
    StatusBar,
    StyleSheet,
    Text,
    View,
} from 'react-native';
import { SafeAreaView } from 'react-native-safe-area-context';
import { filterByCategory } from '../data/products';
import { CategoryBar, DropCard } from '../molecules';

const { width: SCREEN_WIDTH } = Dimensions.get('window');

// Optimized item components
const HeroCard = memo(({ item, onPress }) => (
  <View style={styles.heroWrapper}>
    <DropCard item={item} onPress={onPress} isHero />
  </View>
));

const GridRow = memo(({ items, onPress }) => (
  <View style={styles.gridRow}>
    {items.map(item => (
      <DropCard key={item.id} item={item} onPress={onPress} />
    ))}
    {/* Add placeholder if odd number of items */}
    {items.length === 1 && <View style={styles.placeholder} />}
  </View>
));

// Transform flat list data into hybrid structure
const transformDataForHybridList = data => {
  const result = [];
  let regularItems = [];
  let itemIndex = 0;

  data.forEach((item, index) => {
    const isHero = (index + 1) % 5 === 0;

    if (isHero) {
      // Flush regular items as grid rows first
      while (regularItems.length >= 2) {
        result.push({
          type: 'grid',
          id: `grid-${itemIndex}`,
```

```
          items: regularItems.splice(0, 2),
        });
        itemIndex++;
      }
      // Add remaining single item if exists
      if (regularItems.length === 1) {
        result.push({
          type: 'grid',
          id: `grid-${itemIndex}`,
          items: regularItems.splice(0, 1),
        });
        itemIndex++;
      }
      // Add hero item
      result.push({
        type: 'hero',
        id: item.id,
        item: item,
      });
    } else {
      regularItems.push(item);
      // Create grid row when we have 2 items
      if (regularItems.length === 2) {
        result.push({
          type: 'grid',
          id: `grid-${itemIndex}`,
          items: regularItems.splice(0, 2),
        });
        itemIndex++;
      }
    }
  });

  // Handle remaining items
  while (regularItems.length > 0) {
    const rowItems = regularItems.splice(0, 2);
    result.push({
      type: 'grid',
      id: `grid-${itemIndex}`,
      items: rowItems,
    });
    itemIndex++;
  }

  return result;
};

const FeedScreen = ({ navigation }) => {
  const [selectedCategory, setSelectedCategory] = useState('all');
  const [refreshing, setRefreshing] = useState(false);
```

```

  const filteredProducts = useMemo(() => {
    return filterByCategory(selectedCategory);
  }, [selectedCategory]);

  const transformedData = useMemo(() => {
    return transformDataForHybridList(filteredProducts);
  }, [filteredProducts]);

  const handleProductPress = useCallback(
    item => {
      navigation.navigate('ProductDetail', { product: item });
    },
    [navigation],
  );

  const handleCategoryChange = useCallback(category => {
    setSelectedCategory(category);
  }, []);

  const handleRefresh = useCallback(() => {
    setRefreshing(true);
    // Simulate a refresh delay
    setTimeout(() => {
      setRefreshing(false);
    }, 1500);
  }, []);

  const renderItem = useCallback(
    ({ item }) => {
      if (item.type === 'hero') {
        return <HeroCard item={item.item} onPress={handleProductPress} />;
      }
      return <GridRow items={item.items} onPress={handleProductPress} />;
    },
    [handleProductPress],
  );

  const keyExtractor = useCallback(item => item.id, []);

  const getItemLayout = useCallback((data, index) => {
    // Approximate heights for virtualization optimization
    const HERO_HEIGHT = 380;
    const GRID_ROW_HEIGHT = 260;

    let offset = 0;
    for (let i = 0; i < index; i++) {
      offset += data[i]?.type === 'hero' ? HERO_HEIGHT : GRID_ROW_HEIGHT;
    }

```

```
    const height = data[index]?.type === 'hero' ? HERO_HEIGHT : GRID_ROW_HEIGHT;

    return { length: height, offset, index };
  }, []);

  const ListHeader = useMemo(
    () => (
      <View style={styles.header}>
        <View style={styles.titleContainer}>
          <Text style={styles.logo}>HYPED</Text>
          <Text style={styles.tagline}>Exclusive Drops</Text>
        </View>
        <CategoryBar
          selectedCategory={selectedCategory}
          onCategoryChange={handleCategoryChange}
        />
        <View style={styles.statsBar}>
          <Text style={styles.statsText}>
            {filteredProducts.length} drops available
          </Text>
        </View>
      </View>
    ),
    [selectedCategory, handleCategoryChange, filteredProducts.length],
  );

  return (
    <SafeAreaView style={styles.container} edges={['top']}>
      <StatusBar barStyle="dark-content" backgroundColor="#FFFFFF" />
      <FlatList
        data={transformedData}
        renderItem={renderItem}
        keyExtractor={keyExtractor}
        ListHeaderComponent={ListHeader}
        contentContainerStyle={styles.listContent}
        showsVerticalScrollIndicator={false}
        refreshControl={
          <RefreshControl
            refreshing={refreshing}
            onRefresh={handleRefresh}
            tintColor="#000000"
            colors={['#000000']}
          />
        }
        // Performance optimizations for 500+ items
        initialNumToRender={8}
        maxToRenderPerBatch={10}
        windowSize={11}
        removeClippedSubviews={true}
        updateCellsBatchingPeriod={50}
```

```
        getItemLayout={getItemLayout}
      />
    </SafeAreaView>
  );
};

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#FAFAFA',
  },
  header: {
    backgroundColor: '#FFFFFF',
    paddingBottom: 8,
    marginBottom: 8,
    borderBottomLeftRadius: 24,
    borderBottomRightRadius: 24,
    shadowColor: '#000',
    shadowOffset: { width: 0, height: 2 },
    shadowOpacity: 0.05,
    shadowRadius: 8,
    elevation: 2,
  },
  titleContainer: {
    paddingHorizontal: 20,
    paddingTop: 12,
    paddingBottom: 4,
  },
  logo: {
    fontSize: 36,
    fontWeight: '900',
    color: '#000000',
    letterSpacing: -1,
  },
  tagline: {
    fontSize: 14,
    color: '#888888',
    fontWeight: '500',
    marginTop: 2,
  },
  statsBar: {
    paddingHorizontal: 20,
    paddingVertical: 8,
  },
  statsText: {
    fontSize: 13,
    color: '#666666',
    fontWeight: '500',
  },
  listContent: {
```

```
    paddingBottom: 100,
  },
  heroWrapper: {
    paddingHorizontal: 8,
  },
  gridRow: {
    flexDirection: 'row',
    paddingHorizontal: 8,
    justifyContent: 'space-between',
  },
  placeholder: {
    width: (SCREEN_WIDTH - 48) / 2,
    marginHorizontal: 8,
  },
});

export default FeedScreen;

```